Conferencia 9: Programación Declarativa

Lic. Hiram Borbolla Hernández

Tema: Estrategia de evaluación perezosa. Definición de tipos y clases de tipos.

Estrategias de reducción

La ejecución de un programa funcional consiste en la evaluación de una expresión inicial de acuerdo con las ecuaciones de definición de las funciones y algún *mecanismo de reducción*. La *reducción* es el proceso por el cual, mediante una secuencia de pasos, se transforma la expresión inicial, compuesta de símbolos de función y de símbolos constructores de datos, en un *valor* (de su tipo), o sea, en una expresión que solo contiene ocurrencias de símbolos constructores. El valor obtenido se considera el resultado de la evaluación.

La secuencia de pasos de reducción a seguir durante la evaluación de una expresión funcional determina cómo y cuándo los argumentos de las funciones son evaluados, cuándo son sustituidos en la función y qué forma toma esa sustitución. La aplicación de dichos pasos depende de la estrategia de reducción empleada. Las estrategias o modos de reducción se dividen en dos grupos básicos, la denominada *evaluación impaciente* (*strict o eager evaluation*) y la *evaluación perezosa* (*non-strict o lazy evaluation*), basadas en cómo los argumentos de una función son manipulados.

Evaluación impaciente

La evaluación impaciente se caracteriza por evaluar la función luego de haber evaluado completamente todos sus argumentos. En términos de aplicación de funciones, dada una expresión compuesta por más de un símbolo de función, la reducción de dicha expresión se realiza aplicando las funciones a sus argumentos, desde adentro hacia afuera y de izquierda a derecha. Por ejemplo, de acuerdo a la definición de la función mult como:

```
mult :: (Int, Int) -> Int
mult (x, y) = x * y
```

la expresión mult (1 + 2, 2 + 3) se reduce bajo una evaluación impaciente de la siguiente manera:

```
mult (1 + 2, 2 + 3)      { aplicando 1er + }
= mult (3, 2 + 3)        { aplicando 2do + }
= mult (3, 5)            { aplicando mult }
= 3 * 5                  { aplicando * }
= 15
```

Evaluación perezosa

La evaluación perezosa se caracteriza por evaluar los argumentos de la función solo si estos son necesarios para computar su valor. En términos de aplicación de funciones, dada una expresión compuesta por más de un símbolo de función, la reducción de dicha expresión se realiza aplicando las funciones a sus argumentos, desde afuera hacia adentro y de izquierda a derecha.

Por ejemplo, la expresión anterior mult (1 + 2, 2 + 3) se reduce bajo una evaluación perezosa de la siguiente manera:

```
mult (1 + 2, 2 + 3)       { aplicando mult }
= (1 + 2) * (2 + 3)       { aplicando 1er + }
= 3 * (2 + 3)             { aplicando 2do + }
= 3 * 5                   { aplicando * }
= 15
```

Terminación

Bajo una estrategia impaciente, la evaluación de cualquier expresión que contenga alguna subexpresión cuya evaluación falla, fallará también, sin importar si el resultado de dicha subexpresión se requiere para computar el valor de la expresión general. La evaluación de la siguiente expresión:

```
length [2+1, 1/0, 5-4]
```

fallará con evaluación impaciente debido a la división por cero que se realiza en el tercer elemento de la lista. Sin embargo, con evaluación perezosa, la expresión length [2+1, 1/0, 5-4] da como resultado el valor 3. Su evaluación no falla pues ninguna de las subexpresiones 2+1, 1/0 y 5-4 se evalúan.

Además de fallar, la estrategia impaciente puede conducir a secuencias de reducción que no terminan en situaciones en las que una evaluación perezosa es capaz de computar un valor. Por ejemplo, considere la función infinito definida recursivamente como:

```
infinito :: Int
infinito = 1 + infinito
```

La evaluación de la expresión infinito resulta en un cómputo infinito, que no termina bajo cualquier modo de evaluación que se emplee:

```
Infinito                  { aplicando infinito }
= 1 + infinito            { aplicando infinito }
= 1 + (1 + infinito)      { aplicando infinito }
= 1 + (1 + (1 + infinito))  { aplicando infinito }
...
```

Sin embargo, considere ahora la expresión fst (1+2, infinito). Mientras que la evaluación impaciente de esta expresión no termina:

```
fst (1+2, infinito)       { aplicando + }
= fst (3, infinito)       { aplicando infinito }
= fst (3, 1 + infinito)   { aplicando infinito }
= fst (3, 1 + (1 + infinito))  { aplicando infinito }
...
```

con una evaluación perezosa, esta termina en solo dos pasos y produce el resultado deseado:

```
fst (1+2, infinito)       { aplicando fst }
= 1+2                     { aplicando + }
= 3
```

Este ejemplo muestra que la evaluación perezosa puede producir un resultado en ocasiones en que la evaluación impaciente no termina. De manera general, se cumple una propiedad importante: si existe una secuencia de evaluación que termina para una expresión dada, entonces la evaluación perezosa terminará y producirá el mismo resultado final para dicha expresión. En este sentido, la evaluación perezosa se dice que es justa y se prefiere a la evaluación impaciente si se desea garantizar la evaluación de una expresión siempre que exista una secuencia de pasos que termine y produzca un resultado.

La evaluación perezosa, además, libera al programador de la preocupación por el orden de evaluación de las expresiones. Un tema recurrente cuando el programador quiere ganar eficiencia es su preocupación por no evaluar aquello que no es absolutamente necesario. Considere, por ejemplo, el siguiente pseudocódigo con características de un lenguaje imperativo:

```
int PruebaYCalcula(int arg) {
   var b = VerificaCondicion();
   if (b == true)
      return Calcula2(arg);
   else return 0 ;
}
```

La evaluación impaciente de la expresión PruebaYCalcula(Calcula1(10)) evaluará la expresión Calcula1(10) independientemente de si la expresión VerificaCondicion() evalúa falso y el argumento arg sea necesario para computar un valor. En cambio, bajo una evaluación perezosa la expresión Calcula1(10) se evaluará solo si la expresión VerificaCondicion() evalúa verdadero.

Número de pasos de reducción

La cantidad de reducciones varía según la estrategia de evaluación que se utilice. Por ejemplo, sea la función "doble" definida en conferencias pasadas como:

```
doble :: Int -> Int
doble x = x + x
```

Bajo una evaluación impaciente, la expresión doble (2 * 3) produce la siguiente secuencia de pasos de reducción:

```
doble (2 * 3)          { aplicando * }
= doble 6              { aplicando doble }
= 6 + 6                { aplicando + }
= 12
```

Sin embargo, usando una evaluación perezosa, computar el valor de esta misma expresión requiere de un paso extra de reducción:

```
doble (2 * 3)          { aplicando doble }
= (2 * 3) + (2 * 3)    { aplicando 1er * }
= 6 + (2 * 3)          { aplicando 2do * }
= 6 + 6                { aplicando + }
= 12
```

dado que al aplicar la función doble a sus argumentos se duplica la expresión (2 * 3) y, por tanto, esta debe ser reducida dos veces.

Este ejemplo muestra que la evaluación perezosa puede requerir más pasos de reducción que la evaluación impaciente, especialmente cuando al menos un argumento se utiliza más de una vez en el cuerpo de la función. De manera general, se cumple que con la evaluación impaciente los argumentos se evalúan solo una vez, pero pueden ser evaluados muchas veces bajo una evaluación perezosa.

Estrategias de evaluación y lenguajes de programación

La mayoría de los lenguajes de programación, fundamentalmente lenguajes imperativos, usan el modo de evaluación impaciente. Según las referencias, esto es por razones de facilidad a la hora de implementarlos y además porque es la estrategia que resulta más fiable en los lenguajes que presentan efectos colaterales.

Considere por ejemplo el siguiente código:

```
var b = true;
int f(int arg) {
   if (b == true){
      b = false;
      return arg;
   }
   else{
      b = true;
      return (arg * 2);
   }
}
```

Esta función no es referencialmente transparente, o sea, para una misma entrada el valor de retorno no es siempre el mismo (e.g. f(1) retorna 1 la primera vez y 2 en un segundo llamado). Esto se debe a que la función presenta efectos colaterales, i.e., su valor de retorno depende de una variable global que se modifica cada vez que se llama a la función. Como consecuencia de este efecto, se tiene que la expresión: doble (f(1) + f(2) + f(3)) con evaluación perezosa produce un resultado diferente al que se obtiene con evaluación impaciente, $18 \neq 16$.

Los primeros lenguajes funcionales (Lisp (puro), FP, ML, Hope) también fueron implementados usando una estrategia de evaluación impaciente, no solo por su fácil implementación sino además debido a la diferencia en cuanto a número de pasos de reducción que cada estrategia podría requerir para computar el valor de una misma expresión (e.g doble (2 * 3)). Sin embargo, desde que existen técnicas eficientes de implementación de la *reducción en grafos*, los lenguajes funcionales modernos como Haskell y Miranda utilizan estrategias de evaluación perezosa. En este tipo de implementación, la expresión (2 * 3) del ejemplo anterior solo se evaluaría una vez, ya que sus dos apariciones dentro de la expresión (2 * 3) + (2 * 3) se ubicarían en un único nodo del grafo de reducción. Esta reducción es posible debido a la propiedad de transparencia referencial que cumplen las funciones en los lenguajes funcionales.

La evaluación perezosa es empleada por defecto en varios lenguajes funcionales como Miranda, Clean y Haskell. Esto significa que estos lenguajes garantizan la computación del valor de una expresión siempre que se pueda y, además, debido a su implementación mediante una técnica de reducción en grafos, la cantidad de pasos de reducción nunca es superior a los empleados por una evaluación impaciente.

En otros lenguajes funcionales, la evaluación de las expresiones puede ser demorada de forma explícita empleando sintaxis específicas, como por ejemplo delay y force de Scheme, lazy y Lazy.force en OCaml. En F#, debido a que no es un lenguaje puramente funcional y en sus funciones se pueden introducir efectos colaterales, la estrategia por defecto es la impaciente y para demorar la evaluación de una expresión se emplea la palabra reservada lazy y se fuerza explícitamente su evaluación con la función force del módulo Lazy.

Evaluación perezosa en PF (Haskell)

Corto-circuito gratis

En muchos lenguajes de programación el operador lógico *or* requiere de un tratamiento especial, de modo que su evaluación sea en corto-circuito si el operando izquierdo evalúa a verdadero. En Haskell, este operador (||) es una función como cualquier otra pues la evaluación perezosa inserta esta capacidad en el lenguaje.

De manera general, definir una función en Haskell que se comporte en corto-circuito es muy fácil, por ejemplo:

```
miOr a b = if a then a else b
```

Si se evaluara la expresión miOr True (length [1..] > 0), no se producirá la evaluación del segundo argumento de la función, ya que no resulta necesario para obtener el valor de la expresión. Sin embargo, esta misma función, en un lenguaje con evaluación impaciente, evaluaría los dos argumentos antes de pasarlos a la función, de modo que intentaría calcular la longitud de una lista infinita y produciría un cómputo infinito.

Manejo de estructuras infinitas

Una de las facilidades que brinda la estrategia de evaluación perezosa es la programación con estructuras infinitas, aunque a primera vista pareciera imposible. Considere las siguientes funciones:

```
ones :: [Int]
ones = 1 : ones

take :: Int -> [a] -> [a]
take 0 _ = []
take (n + 1) [] = []
take (n + 1) (x : xs) = x : take n xs
```

La función ones define una lista infinita de 1*s*, mientras que la función take extrae los *n* primeros elementos de una lista.

La evaluación de la función ones no termina, pues por definición construye una lista infinita de números 1. Sin embargo, esta puede utilizarse en conjunción con otras funciones en expresiones que bajo una evaluación perezosa sí terminan. Por ejemplo, la expresión take 2 ones termina y se reduce de la siguiente manera:

```
take 2 ones                 { aplicando ones }
= take 2 (1 : ones)         { aplicando take }
= 1 : take 1 ones           { aplicando ones }
= 1 : take 1 (1 : ones)     { aplicando take }
= 1 : 1 : take 0 ones       { aplicando take }
= 1 : 1 : []                { notación de listas }
= [1, 1]
```

Eficiencia

La evaluación perezosa da lugar a que definiciones aparentemente ineficientes no lo sean. Considere el siguiente ejemplo, donde se define el algoritmo de ordenación por inserción y, a partir de este, una función que determina el mínimo de una lista.

```
ins :: Ord a => a -> [a] -> [a]
ins x [] = [x]
ins x (y : ys)  | x <= y = x : y : ys
                | otherwise = y : ins x ys

ordIns :: Ord a => [a] -> [a]
ordIns [] = []
ordIns (x : xs) = ins x (ordIns xs)

min xs = head (ordIns xs)
```

Bajo una evaluación impaciente, está claro que la definición de min sería muy ineficiente ($O(n^2)$), pues tendría que ordenarse primero la lista para luego obtener su primer elemento. Sin embargo, bajo una evaluación perezosa, la función min es de orden lineal ($O(n)$). Observe la secuencia de pasos de reducción que se realiza al evaluar la expresión min [9, 5, 4, 3, 2]:

```
min [9, 5, 4, 3, 2]                              { aplicando min }
= head (ordIns [9, 5, 4, 3, 2])                  { aplicando ordIns }
= head (ins 9 (ordIns [5, 4, 3, 2]))             { aplicando ordIns }
...
= head (ins 9 (ins 5 (ins 4 (ins 3 (ins 2 [])))))  { aplicando ins }
= head (ins 9 (ins 5 (ins 4 (ins 3 [2]))))       { aplicando ins }
= head (ins 9 (ins 5 (ins 4 (2 : ins 3 []))))    { aplicando ins }
= head (ins 9 (ins 5 (2 : ins 4 (ins 3 []))))    { aplicando ins }
= head (ins 9 (2 : ins 5 (ins 4 (ins 3 []))))    { aplicando ins }
= head (2 : ins 9 (ins 5 (ins 4 (ins 3 []))))    { aplicando head }
= 2
```

Optimización

La propiedad de transparencia referencial que cumplen las funciones en los lenguajes funcionales permite que, una vez que se ha evaluado una expresión, esta se puede reutilizar. La ausencia de efectos colaterales en los programas en combinación con la evaluación perezosa y la técnica de reducción en grafos constituye una herramienta poderosa para la optimización de código, fundamentalmente en los lenguajes funcionales.

Modularidad

Un procedimiento simple para generar la secuencia infinita de todos los números primos es el siguiente:

1. Escribir la secuencia 2, 3, 4, 5, 6,…;
2. Marcar el primer número, p, en la secuencia como primo;
3. Borrar de la secuencia todos los múltiplos de p;
4. Volver al paso 2

Este procedimiento se conoce como la *Criba de Eratóstenes* (*sieve of Eratosthenes*) en honor al matemático griego que lo describió. Como se puede observar, los pasos 1 y 3 del algoritmo requieren de un tiempo infinito e impiden su implementación en lenguajes convencionales. Sin embargo, en Haskell, este puede escribirse de una forma muy simple:

```
primos :: [Int]
primos = cribar [2..]

cribar :: [Int] -> [Int]
cribar (p : xs) = p : cribar [x | x <- xs, x `mod` p /= 0]
```

Si se evalúa la expresión primos se obtiene como es de esperar la secuencia infinita de números primos:

```
Hugs> primos
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67 ,..
```

La evaluación perezosa permite hacer más:

```
Hugs> take 10 primos
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

```
Hugs> takeWhile (<10) primos
[2, 3, 5, 7]
```

Se ha separado la generación de los datos de las partes de control, permitiendo que estos sean utilizados en diferentes situaciones.

Definición de tipos y clases de tipos

Tipos definidos por el usuario

En Haskell los tipos de datos se definen usando la palabra clave data con la siguiente notación:

```
data Tipo = Constructor_1 | Constructor_2 | ... | Constructor_k
```

La parte anterior al símbolo = denota el tipo, mientras que la posterior denota sus constructores, o sea, los diferentes valores que el tipo declarado pudiera tener. El símbolo | se lee como "o" y tanto el nombre del tipo como los constructores deben escribirse en mayúscula.
Por ejemplo, la definición del tipo Bool en el estándar Prelude es la siguiente:

```
data Bool = False | True
```

y se lee como, el tipo Bool puede tener los valores True o False.
De igual forma, se podría definir el tipo Color como:

```
data Color = Rojo | Amarillo | Verde | Azul | Violeta
```

Los constructores en una declaración de tipo pudieran tener argumentos. Por ejemplo, el tipo Figura, que contiene como valores un Circulo o un Rectangulo, puede declararse como:

```
data Figura = Circulo Float | Rectangulo Float Float
```

En este caso, la declaración del tipo Figura plantea que sus valores pueden ser un Circulo, representado por su radio (de tipo Float), y un Rectangulo representado por sus dimensiones (ambos de tipo Float). Note que los constructores son funciones cuyo valor de retorno es del tipo de dato declarado. En el ejemplo, el tipo de Circulo es Circulo :: Float -> Figura y el tipo de Rectangulo es Rectangulo :: Float -> Float -> Figura.

La diferencia fundamental entre las funciones normales y las funciones constructoras de datos es que estas últimas no poseen ecuaciones de definición como las primeras y existen con el único propósito de construir datos. Por ejemplo, mientras la expresión signum -5 puede evaluarse al valor -1 aplicando la ecuación de definición de la función signum, la expresión Rectangulo 4 6 no puede ser reducida pues no existe ecuación de definición para Rectangulo. Dicha expresión ya está completamente evaluada y constituye un valor como lo es el número -5 y el valor booleano True.

Sin embargo, como funciones al fin, pueden usarse donde quiera que se requiera de una función y su tipo lo permita, por ejemplo:

```
Hugs> map Circulo [1 .. 5]
[Circulo 1.0, Circulo 2.0, Circulo 3.0, Circulo 4.0, Circulo 5.0]

Hugs> map area (map Circulo [1 .. 5])
[3.141593, 12.56637, 28.27433, 50.26548, 78.53982]
```

Como los valores de los tipos declarados en el estándar Prelude, estos constructores pueden usarse para definir funciones sobre el tipo Figura. Por ejemplo, se pueden definir las funciones cuadrado y area de la siguiente manera:

```
cuadrado :: Float -> Figura
cuadrado n = Rectangulo n n

area :: Figura -> Float
area (Circulo r) = pi * r ^ 2
area (Rectangulo x y) = x * y
```

Ahora es posible calcular el área de cualquiera de las tres figuras:

```
Hugs> area (Circulo 3)
28.27433

Hugs> area (Rectangulo 3 4)
12.0

Hugs> area (cuadrado 3)
9.0
```

Pero si se intenta evaluar la expresión Rectangulo 3 4 se recibe un error:

```
Hugs> Rectangulo 3 4
ERROR - Cannot find "show" function for:
*** Expression : Rectangulo 3 4
*** Of type      : Figura
```

Esto es porque Haskell todavía no sabe cómo mostrar el nuevo tipo de dato. Como se puede ver en el texto del error, no se encuentra definida la función show para el tipo de dato Figura. Resolver este problema es fácil y consiste en hacer el tipo Figura parte de la clase de tipo Show modificando su declaración de la siguiente manera:

```
data Figura = Circulo Float | Rectangulo Float Float deriving (Show)
```

La expresión deriving (Show) después de los constructores de datos es interpretado por Haskell como que automáticamente defina la función show en función de la definición que hacen de ella las distintas partes de los constructores, en este caso, el show del tipo String y el del tipo Float.

Ahora es posible evaluar la expresión Rectangulo 3 4 que, como está evaluada, lo único que hace es imprimir la expresión utilizando la función show:

```
Hugs> Rectangulo 3 4
Rectangulo 3.0 4.0
```

Tipos de datos polimórficos

Las declaraciones de tipos de datos pueden contener variables de tipo. Por ejemplo:

```
data Par a = P a a
```

Debido a la ocurrencia de una variable de tipo en su declaración, se dice que “Par a” es un constructor de tipo, pues a partir de él se pueden construir otros tipos, por ejemplo, Par Bool, Par [Char], Par Float, etc.

```
P True False :: Par Bool
P "Hola" "Mundo" :: Par [Char]
P 2.0 3.0 :: Par Float
```

El tipo Par es un ejemplo de tipo polimórfico y sobre él se pueden definir funciones polimórficas, como:

```
fst  (P a _) = a
snd (P _ a) = a
```

cuyas definiciones de tipo son:

```
fst  :: Par a -> a
snd :: Par a -> a
```

Records

Supóngase que se desea crear el tipo de dato Persona con la información de su nombre, su apellido, su edad y su ocupación. Una forma de hacerlo es la siguiente:

```
data Persona = Persona String String Int String deriving (Show)
```

La cual permite crear datos de tipo Persona de la siguiente manera:

```
Hugs> Persona "Julia" "Sanchez" 30 "Contadora"
Persona "Julia" "Sanchez" 30 "Contadora"

Hugs> Persona "Pedro" "Gonzalez" 35 "Medico"
Persona "Pedro" "Gonzalez" 35 "Medico"
```

Supóngase ahora que se desea extraer información de una persona, como su nombre, su apellido, etc. Esto puede hacerse definiendo funciones (como fst y snd) que separan la información, por ejemplo:

```
nombre :: Persona -> String
nombre (Persona n _ _ _) = n

apellido :: Persona -> String
apellido (Persona _ a _ _) = a

edad :: Persona -> Int
edad (Persona _ _ e _) = e

ocupacion :: Persona -> String
ocupacion (Persona _ _ _ o) = o
```

Aunque no es complicado de hacer y funciona, resulta agotador y algo aburrido tener que definir una función para cada uno de los campos que contiene el tipo persona. Afortunadamente, existe una sintaxis especial para declarar tipos de datos cuyos campos son de interés extraer: los *records*. El tipo de dato Persona usando records se definiría de la siguiente manera:

```
data Persona = Persona { nombre :: String,
                         apellido :: String,
                         edad :: Int,
                         ocupacion :: String } deriving (Show)
```

Una característica de los records es que se pueden crear personas proporcionando los campos en orden diferente a como fueron definidos, con la sintaxis campo = valor, por ejemplo:

```
Hugs> Persona {ocupacion = "Contadora", apellido = "Sanchez", edad = 30, nombre = "Julia"}
Persona {nombre = "Julia", apellido = "Sanchez", edad = 30, ocupacion = "Contadora"}
```

De la manera usual también:

```
Hugs> Persona "Pedro" "Gonzalez" 35 "Medico"
Persona {nombre = "Pedro", apellido = "Gonzalez", edad = 35, ocupacion = "Medico"}
```

El beneficio principal de usar records es que no es necesario crear funciones para extraer la información correspondiente a una persona. Las funciones nombre, apellido, edad y ocupacion se crean automáticamente.

```
Hugs> :t nombre
nombre :: Persona -> String

Hugs> :t apellido
apellido :: Persona -> String
```

Tipos recursivos

En la declaración de un tipo de dato no hay restricción para que los constructores sean de un tipo diferente al tipo que se está declarando. Por tanto, es posible la definición de tipos de datos recursivos. Por ejemplo, el tipo Expr:

```
data Expr = Lit Integer | Suma Expr Expr | Resta Expr Expr
```

Luego, es posible definir un evaluador de expresiones:

```
eval :: Expr -> Integer
eval (Lit n) = n
eval (Suma e1 e2) = eval e1 + eval e2
eval (Resta e1 e2) = eval e1 - eval e2
```

Además de las listas, una estructura de datos recursiva muy utilizada en programación es el árbol. Una forma de declarar esta estructura es la siguiente:

```
data Arbol a = Vacio | Nodo a (Arbol a) (Arbol a)
```

Dado que en la declaración se usa una variable de tipo, el tipo Arbol es polimórfico, luego se pueden construir árboles de diferentes tipos. Por ejemplo:

```
Nodo 'a' (Nodo 'b' Vacio (Nodo 'd' Vacio Vacio)) (Nodo 'e' Vacio Vacio) :: Arbol Char
Nodo 1 (Nodo 2 Vacio Vacio) (Nodo 3 Vacio Vacio) :: Num a => Arbol a
```

También se pueden definir funciones polimórficas sobre árboles, como la función contiene que determina si un elemento está contenido en el árbol y la función elems que obtiene la lista con todos los elementos que contiene el árbol.

```
contiene :: Eq a => Arbol a -> a -> Bool
contiene Vacio _ = False
contiene (Nodo a b c) e | a == e = True
                        | otherwise = contiene b e || contiene c e
```

```
elems :: Arbol a -> [a]
elems Vacio = []
elems (Nodo a b c) = elems b ++ [a] ++ elems c
```

Definición de Clases de tipos

Para definir una clase de tipo se utiliza la palabra clave class con la notación siguiente:

```
class Nombre a where
    funcion1 :: tipo_funcion1
    funcion2 :: tipo_funcion2
    ...
    funcionk :: tipo_funcionk
```

donde Nombre representa el nombre de la clase de tipo que se está definiendo y "a" es una variable de tipo que denota cualquier tipo que se haga instancia de la clase. Después del where se escribe el cuerpo de la clase, i.e., las declaraciones de tipo de las funciones que definen su comportamiento. Además de las declaraciones de tipo, las clases pueden definir un comportamiento por defecto para cada una de sus funciones. Por ejemplo, la clase Eq se define en el estándar Prelude de la siguiente manera:

```
class Eq a where
    (==) :: a -> a -> Bool
    (/=) :: a -> a -> Bool
     x == y = not (x /= y)
     x /= y = not (x == y)
```

La definición por defecto de la operación de comparación (==) en función de (/=), y viceversa, permite, al declarar un tipo como instancia de la clase Eq, que solo sea necesario definir una de las dos funciones. No obstante, todas las definiciones por defecto que hacen las clases de tipo de sus funciones pueden sobrescribirse. En algunos casos, pudiera existir alguna manera más eficiente de comprobar que dos valores son diferentes que simplemente chequear si estos no son iguales.

Declaración de instancia de tipo

La declaración de un tipo como instancia de una clase se hace utilizando la siguiente notación:

```
instance Clase Tipo where
     definicion_funcion1
     definicion_funcion2
     ...
     definicion_funcionk
```

donde Clase denota la clase de la cual se quiere hacer instancia el tipo "Tipo". Por ejemplo, el tipo Bool se ha declarado en el Prelude como instancia de la clase Eq de la siguiente manera:

```
instance Eq Bool where
     False == False = True
     True == True = True
     _ == _ = False
```

Por razones técnicas, solo los tipos declarados con data pueden hacerse instancias de clases de tipos.

Instancias derivadas

Cuando se crean nuevos tipos, con frecuencia se desea declarar estos como instancias de algunas clases de tipos predefinidas. Como se adelantó en uno de los ejemplos anteriores, Haskell provee una forma sencilla y automática de declarar nuevos tipos como instancias de las clases Eq, Ord, Show y Read mediante el mecanismo de derivación deriving. Por ejemplo, en el Prelude el tipo Bool está declarado de la siguiente manera:

```
data Bool = False | True deriving (Eq, Ord, Show, Read)
```

lo cual indica que este sea un tipo válido para las funciones que definen estas clases, así como para aquellas que poseen restricciones de estas clases de tipo.

En el caso de tipos cuyos constructores poseen argumentos, el tipo de cada argumento también debe ser instancia de cualquier clase derivada. Por ejemplo, si se desea además derivar el tipo Figura, declarado anteriormente, de la clase Eq es preciso que el tipo Float también sea instancia de esta clase, lo cual es cierto. Luego, es posible modificar su declaración de la siguiente manera:

```
data Figura = Circulo Float | Rectangulo Float Float deriving (Show, Eq)
```

Así, dos figuras pueden compararse en cuanto a igualdad o desigualdad:

```
Hugs> Rectangulo 3 4 == Rectangulo 3 4
True

Hugs> Circulo 3 == Circulo 4
False

Hugs> Circulo 3 \= Circulo 4
True
```